

План дальнейшей разработки `requisites_extractors`

Документ разбит на 9 независимых частей (эпиков). Каждая часть → отдельная ветка/набор коммитов.

Внутри каждой части — пошаговые задачи с критериями готовности (DoD).

Порядок частей выстроен по приоритету: сначала то, что даёт быстрый прирост качества и устраняет долги, потом расширение функционала.

Текущее состояние (аудит на 14.06.2026)

Что уже работает:

- Полный pipeline в `app/services/pipeline_service.py`: `routing` → `extraction` → `normalization` → LLM → `fallback regex` → `validation` → `export`.
- Маршрутизация форматов (`routing_service.py`): `DOCX` / `PDF-text` / `PDF-scan` / `image` / `unsupported`.
- Экстракторы: `docx_extractor`, `pdf_text_extractor`, `pdf_ocr_extractor`, `image_ocr_extractor` (Tesseract).
- Валидаторы с контрольными суммами: ИНН, КПП, ОГРН, БИК, счета + кросс-проверка БИК ↔ `корр.счёт`.
- LLM-слой: единый интерфейс + `mock`, `openai`, `ollama` клиенты, версионирование промптов.
- `Fallback regex`-слой и `merge LLM+regex` с трекингом `extracted_by`.
- Экспортёры: `JSON`, `XLSX`, заполнение шаблона `DOCX`.
- Web UI на `Flask` (загрузка, результат, скачивание).

Технический долг (главные проблемы):

1. Нет тестов вообще — папки `tests/` не существует. Это блокирует рефакторинг.
2. Отладочные `print()` прямо в `pipeline_service.py` (шаги 6 и 7) — мусор в проде.
3. Пустые файлы-заглушки: `app/api/*`, `app/main.py`, `app/dependencies.py`, `ingest_service.py`, `postprocess_service.py`, `llm_extraction_service.py`, `export_service.py`, `forms.py`, `cross_field_validator.py`, OCR backends (`ocr/base.py`, `tesseract_backend.py`, `easyocr_backend.py`), `scripts/batch_process.py`, `scripts/run_dev.py`.
4. `README` пустой, нет инструкции запуска.
5. Секреты в репозитории: `.env` закоммичен, нет `.env.example`.
6. Один монолитный коммит `first-commit` — нет истории.

7. **Мусор в репозитории:** `123.txt`, `script.py`, `fix_structure.py`, `test_real.docx`, `.idea/`, заметки `.txt`, 90+ файлов в `exports/`.
8. Нет Docker / CI / линтеров, хотя в навыках заявлены.
9. OCR изображений примитивный: только `image_to_string`, нет `image_to_data`, нет сборки таблиц (это уже описано в ваших заметках от 12.06).

Часть 1. Гигиена репозитория и базовая инфраструктура

Цель: привести репозиторий в чистый, безопасный, воспроизводимый вид. Без этого опасно делать дальнейшие изменения.

Шаги

1. Почистить мусор из git.

- Удалить из репозитория: `123.txt`, `script.py`, `fix_structure.py`, `test_real.docx`, `12.06.26 продолжение программы.txt`, `план.txt` (переместить полезное в `docs/`), `.idea/`.
- Удалить содержимое `exports/` из git (оставить только `.gitkeep`).

2. Исправить `.gitignore`.

- Добавить: `.env`, `.idea/`, `__pycache__`, `/*.pyc`, `venv/`, `.venv/`, `exports/*`, `processed/*`, `uploads/*`, `logs/*.log`, `*.docx` (кроме `shablon.docx`).

3. Убрать секреты.

- Удалить `.env` из git-истории (`git rm --cached .env`).
- Создать `.env.example` со всеми ключами из `config.py`, но без значений.
- ⚠ Если в `.env` лежал реальный `OPENAI_API_KEY` — отозвать его, т.к. он попал в публичную историю.

4. Разбить `requirements`.

- `requirements/base.txt` (runtime), `requirements/dev.txt` (pytest, ruff, black, mypy), `requirements/prod.txt`.
- Убрать дубль `flask` (он указан дважды).

5. Заполнить `pyproject.toml` — добавить конфиги `ruff`, `black`, `mypy`, `pytest`.

DoD: `git clone` даёт чистый репозиторий без секретов и мусора; `pip install -r requirements/dev.txt` ставит всё нужное.

Часть 2. Чистка кода и удаление заглушек

Цель: убрать отладочный мусор и определиться с пустыми файлами.

Шаги

1. Удалить отладочные `print()` из `pipeline_service.py` (шаги 6 и 7, ~50 строк). Заменить на `logger.debug(...)` с тем же содержимым (под флагом, который выключен в проде).
2. Решить судьбу пустых файлов:
 - Либо реализовать (если в плане), либо удалить, чтобы не вводить в заблуждение.
 - `cross_field_validator.py` — кросс-проверки уже частично в `account_validator`; либо перенести туда логику, либо удалить файл.
 - `ingest_service.py`, `postprocess_service.py`, `llm_extraction_service.py`, `export_service.py` — логика уже размазана по `pipeline`; решить: выносить в эти сервисы (чище) или удалить.
3. Вынести магические числа (`12_000`, `ocr_min_text_chars`) в `config.py` / `constants.py`.
4. Прогнать `ruff + black` по всему `app/`, исправить замечания.

DoD: в проде нет ни одного `print()`; `ruff check app/` без ошибок; нет пустых модулей, которые импортируются.

Часть 3. Тесты (фундамент для всего остального)

Цель: покрыть тестами критичную бизнес-логику. Это самая важная часть — после неё рефакторинг становится безопасным.

Шаги

1. Создать структуру `tests/ + conftest.py + tests/fixtures/`.
2. Unit-тесты валидаторов (быстрый и надёжный выигрыш):
 - `test_validators_inn.py` — валидные 10/12-значные, битая контрольная сумма, буквы, пустое.
 - `test_validators_kpp.py`, `test_validators_ogrn.py` (13/15), `test_validators_bik.py`, `test_validators_accounts.py`.
 - Кросс-проверка БИК ↔ корр.счёт.
 - Использовать реальные валидные тестовые реквизиты (например, известные публичные ИНН/ОГРН).
3. Unit-тесты `regex`-слоя `test_fallback_regex.py` — ИНН/КПП-связка, ОГРН, email, phone, p/c, k/c, БИК с разными разделителями и OCR-шумом.
4. Unit-тесты нормализации `test_normalization.py`.
5. Unit-тесты `routing` `test_routing.py` — по расширению/MIME/контенту PDF.
6. Интеграционные тесты `pipeline` на `mock-LLM`:
 - `test_pipeline_docx.py`, `test_pipeline_pdf_text.py` — на 2–3 эталонных документах в `fixtures/`.
 - Проверять: заполненность полей, `needs_review`, наличие экспортов.

7. Настроить `pytest` (coverage порог, например 70% на `validators/` и `services/`).

DoD: `pytest` зелёный; покрытие валидаторов $\geq 90\%$; есть фикстуры с эталонными документами.

Часть 4. CI/CD (GitHub Actions)

Цель: автоматическая проверка на каждый `push/PR`. Вы изучали CI/CD — это прямое применение.

Шаги

1. `.github/workflows/ci.yml`:
 - Matrix по Python (3.11, 3.12).
 - Шаги: `install` $\rightarrow$ `ruff check` $\rightarrow$ `black --check` $\rightarrow$ `myru app` $\rightarrow$ `pytest --cov`.
 - Кэш `pip`-зависимостей.
2. Бейджи в README (build status, coverage).
3. (Опционально) `pre-commit hooks` — `pre-commit-config.yaml` с `ruff/black`, чтобы ловить до коммита.
4. (Опционально) `Dependabot` для обновления зависимостей.

DoD: PR не мёржится с красным CI; бейдж зелёный в README.

Часть 5. Качество извлечения: усиление OCR и промптов

Цель: реализовать то, что описано в ваших заметках от 12.06 — это даёт самый заметный прирост качества на «карточках контрагента».

Шаги (порядок = порядок коммитов из ваших заметок)

1. `Validator hardening` — расширить `validation_service`: возвращать `review_reasons`, помечать «`ogrn looks like classifier code`» (ОКПО/ОКТМО/ОКВЭД $\neq$ ОГРН), очищать невалидные поля в `None` с `warning`.
2. Усилить `fallback regex` — искать БИК/р/с/к/с рядом с ключом (а не любое 9/20-значное число); канонизация (р/с начинается с 40, к/с с 30).
3. `Split промптов` — `build_text_prompt()` и `build_image_table_prompt()`; отдельный профиль для `image/table` с жёсткими правилами (ИНН/КПП в разные поля, ОГРН ровно 13 цифр и т.д.).
4. `Нормализация OCR-текста` — `normalize_ocr_text`, `normalize_requisite_numbers`, `split_classifiers_block` (вынести блок классификаторов ОКПО/ОКАТО/ОКТМО/ОКОГУ/ОКВЭД).
5. `Структурный OCR для изображений` — перейти на `pytesseract.image_to_data` (`Output.DICT`), `group_words_into_lines()` по `top`-координате, `extract_key_value_lines()`; сохранять `debug`-артефакты (`raw_ocr_text`, `ocr_lines`, `kv_candidates`).

6. Реализовать OCR backends — `ocr/base.py` (интерфейс), `tesseract_backend.py`, `easyocr_backend.py`; переключение через `settings.ocr_backend`.

7. Расширить схему результата — `extracted_by`, `review_reasons`, `fill_rate` в JSON-экспорте (часть уже есть).

DoD: на тест-наборе «карточек контрагента» доля верно извлечённых ИНН/КПП/ОГРН/БИК/счетов заметно растёт; есть метрика `fill_rate` до/после.

Часть 6. REST API (Flask)

Цель: реализовать пустые `app/api/*` — отдельный JSON-слой рядом с Web UI.

Шаги

1. `app/api/schemas_http.py` — Pydantic-модели запроса/ответа API.

2. `app/api/routes_health.py` — GET `/api/health` (статус, версия).

3. `app/api/routes_upload.py` — POST `/api/extract` (multipart файл → JSON-результат).

4. `app/main.py` — фабрика Flask-приложения (`create_app`), регистрация blueprints (`web + api`), конфиг, error handlers.

5. `app/dependencies.py` — общие зависимости (билдер pipeline, лимиты).

6. Единый формат ошибок — JSON `{error, code, details}` через `app/core/exceptions.py`.

7. Тесты API — `test_api.py` (health, успешный extract, невалидный файл, 413 на превышение размера).

DoD: `curl -F file=@sample.docx localhost:5000/api/extract` возвращает корректный JSON; есть тесты эндпоинтов.

Часть 7. CLI и batch-обработка

Цель: реализовать заявленные режимы запуска (CLI приоритетен по вашему же плану).

Шаги

1. Дописать `scripts/run_dev.py` — запуск Flask в dev-режиме.

2. `scripts/run_cli.py` — уже 214 строк; провести ревизию, привести к актуальному pipeline, добавить тест.

3. `scripts/batch_process.py` — обработка всех файлов из папки: прогресс-бар, сводный отчёт (CSV/JSON), счётчик `needs_review`.

4. Единая точка входа — `python -m app` или `console_scripts` в `pyproject.toml`.

DoD: `python scripts/batch_process.py ./input_dir` обрабатывает папку и выдаёт сводку.

Часть 8. Контейнеризация (Docker)

Цель: воспроизводимый запуск, в т.ч. с системными зависимостями OCR (tesseract, poppler).

Шаги

1. Dockerfile — multi-stage; установить tesseract-ocr, tesseract-ocr-rus, poppler-utils, libmagic; non-root user.
2. docker-compose.yml — сервис app + (опционально) Ollama для локального LLM.
3. .dockerignore — исключить exports/, venv/, .git/, tests/.
4. Healthcheck в compose на /api/health.
5. Документировать запуск через Docker в README.

DoD: docker compose up поднимает рабочее приложение с OCR без ручной установки зависимостей.

Часть 9. Документация и наблюдаемость

Цель: проект должен быть понятен любому новому разработчику и пригоден для отладки.

Шаги

1. README.md — описание, архитектура (диаграмма pipeline), быстрый старт (local/Docker), переменные окружения, примеры CLI/API, список полей реквизитов.
2. docs/ — перенести план.txt и заметки от 12.06 как docs/architecture.md и docs/extraction-notes.md.
3. CONTRIBUTING.md — как запускать тесты, линтеры, формат коммитов.
4. Структурное логирование — request/document id во всех логах (частично есть), время по этапам, причина needs_review.
5. (Опционально) метрики — счётчик обработанных документов, средний fill_rate, доля needs_review.

DoD: новый разработчик поднимает проект по README за < 15 минут.

Рекомендуемый порядок выполнения

Приоритет	Часть	Почему сейчас
1	Ч.1 Гигиена репо	Безопасность (секреты!) и чистая база
2	Ч.2 Чистка кода	Убрать print() из прода
3	Ч.3 Тесты	Фундамент для безопасного рефакторинга
4	Ч.4 CI/CD	Автопроверка качества

Приоритет	Часть	Почему сейчас
5	Ч.5 Качество извлечения	Главный продуктовый прирост
6	Ч.6 REST API	Расширение интерфейсов
7	Ч.7 CLI/batch	Завершение режимов запуска
8	Ч.8 Docker	Воспроизводимость
9	Ч.9 Документация	Финальная полировка

Совет: Части 1–4 можно сделать быстро (1–2 вечера каждая) — это «расчистка» перед основной работой. Часть 5 — основная продуктовая ценность, её стоит дробить на 7 отдельных коммитов (по шагам). Части 6–9 — независимы и могут идти параллельно.

Срочное (сделать в первую очередь)

1. **△ Проверить**, был ли реальный API-ключ в закоммиченном `.env`. Если да — отозвать его немедленно (он в git-истории навсегда).
2. **Убрать** `print()` из `pipeline_service.py` — 5 минут, заметный эффект.
3. **Создать** `tests/` хотя бы для валидаторов — это даёт уверенность при любых правках.